BSc Hons in Computing

2024.2

Problem Solving with Data Structures and Algorithms (PDSA)

Individual Report: Minimum Cost Assignment Problem

Coursework 1

Coventry Index : 16110762

NIBM Registered Name : E M A S B EKANAYAKA

Faculty of Engineering, Environment and Computing

School of Computing, Electronics and Mathematics

Coventry University

School of Computing and Engineering

National Institute of Business Management

List of Figures

1 Algorithm timing across game rounds (Greedy vs. Hungarian). 11
2 Rows of the `algorithm_results` table produced during game play. . . 13

List of Tables

1 Complexity summary from program logic and observed outputs 8
2 Comparison of Greedy and Hungarian algorithms 10
3 Column descriptions for the `algorithm_results` table 14

Contents

Introduction **1**

1 Program Logic **2**
1.1 Game Round Generation 2
1.2 Greedy Assignment Algorithm 2
1.3 Hungarian Algorithm 3
1.4 Timing Measurement 5
1.5 Database Persistence 6

2 Complexity Analysis **7**
2.1 Greedy Algorithm – $O(n^2)$ Time, $O(n)$ Space 7
2.1.1 Time Complexity 7
2.1.2 Space Complexity 7
2.1.3 Optimality 7
2.2 Hungarian Algorithm – $O(n^3)$ Time, $O(n^2)$ Space 7
2.2.1 Time Complexity 7
2.2.2 Space Complexity 8
2.2.3 Optimality 8
2.3 Analysis Based on Program Outputs 8

3 Comparison of Algorithmic Approaches **9**
3.1 Approach and Design Philosophy 9
3.2 Cost Quality 9
3.3 Runtime Performance 9
3.4 Summary Comparison 10
3.5 When to Prefer Each Algorithm 10

4 Timing Chart – Game Rounds **11**

5 Video Demonstration **12**

6 Database Output **13**

List of References **15**

Introduction

This report presents an individual analysis of the Minimum Cost Assignment game implemented as part of the PDSA coursework. The application is a full-stack web game built with a Python FastAPI backend, a React frontend, and an SQLite database.

The core problem is the **Assignment Problem**: given N employees and N tasks, where assigning employee i to task j incurs a cost c_{ij}, find a one-to-one assignment that minimises the total cost. In each game round, N is chosen randomly in the range $[50, 100]$ and each cost c_{ij} is drawn randomly from $[\$20, \$200]$.

Two algorithms are implemented and compared: a **Greedy** heuristic and the **Hungarian algorithm** (Kuhn–Munkres method).

1. Program Logic

1.1 Game Round Generation

Each round begins when the frontend calls the `GET /api/round/new` endpoint. The backend uses Python's `random` module to produce a random problem instance:

```python
@app.get("/api/round/new", response_model=NewRoundResponse)
def new_round():
    n = random.randint(50, 100)
    cost_matrix = [
        [random.randint(20, 200) for _ in range(n)]
        for _ in range(n)
    ]
    return NewRoundResponse(n=n, cost_matrix=cost_matrix)
```

Listing 1: Game round generation – backend/main.py

The $N \times N$ cost matrix is sent to the frontend, which renders it as an interactive table. The player may optionally assign some tasks manually before requesting the algorithmic solutions.

1.2 Greedy Assignment Algorithm

The greedy approach scans employees in order $0, 1, \ldots, N-1$. For each employee it picks the *cheapest unassigned task* and commits to that choice immediately. Because earlier decisions are never reconsidered, the greedy algorithm does not guarantee a globally optimal assignment; it serves as a fast baseline.

```python
for employee in range(n):
    best_task = -1
    best_cost = float("inf")
    candidates = []

    for task in range(n):
        if task not in assigned_tasks:
            candidates.append((task, cost_matrix[employee][task]))
            if cost_matrix[employee][task] < best_cost:
                best_cost = cost_matrix[employee][task]
                best_task = task

    assignment[employee] = best_task
    assigned_tasks.add(best_task)
    running_total += int(best_cost)

    steps.append({
        "employee":      employee,
        "task":          best_task,
        "cost":          int(best_cost),
        "running_total": running_total,
        "candidates":    sorted(candidates, key=lambda x: x[1])[:5],
    })

total_cost = sum(cost_matrix[emp][assignment[emp]] for emp in range(n))
return assignment, total_cost, steps
```

Listing 2: Greedy core loop – backend/algorithms/greedy.py

The function returns three values:

- `assignment` – a list where `assignment[i]` is the task index allocated to employee i.
- `total_cost` – the sum of all assignment costs in dollars.
- `steps` – a list of dictionaries (one per employee) that the frontend uses to animate the decision process step by step.

1.3 Hungarian Algorithm

The Hungarian algorithm (Kuhn–Munkres, 1955) is guaranteed to find the *globally optimal* assignment. It operates on a reduced cost matrix and uses König's theorem to

determine when an optimal assignment has been found.

The algorithm proceeds through five logical phases that repeat until an optimal solution is identified:

1. **Row reduction.** Subtract the minimum value of each row from every cell in that row, ensuring at least one zero per row.
2. **Column reduction.** Subtract the minimum value of each column from every cell in that column, ensuring at least one zero per column.
3. **Minimum line cover.** Find the minimum number of horizontal or vertical lines that cover all zeros. This is equivalent to computing a maximum bipartite matching on the zero graph via augmenting paths (König's theorem).
4. **Optimality check.** If the number of covering lines equals N, a perfect assignment through zeros exists and is optimal.
5. **Matrix adjustment.** If the cover count is less than N, find the minimum uncovered value δ, subtract δ from all uncovered cells, add δ to doubly-covered cells, and repeat from step 3.

```python
n = len(cost_matrix)
matrix = [row[:] for row in cost_matrix]

# Step 1 -- Row reduction
for i in range(n):
    row_min = min(matrix[i])
    for j in range(n):
        matrix[i][j] -= row_min

# Step 2 -- Column reduction
for j in range(n):
    col_min = min(matrix[i][j] for i in range(n))
    for i in range(n):
        matrix[i][j] -= col_min

# Iterate until minimum line cover equals n
while True:
    row_cover, col_cover = _min_line_cover(matrix, n)
    if len(row_cover) + len(col_cover) >= n:
        break
    _adjust_matrix(matrix, n, row_cover, col_cover)

assignment = _extract_assignment(matrix, n)
total_cost = sum(cost_matrix[emp][assignment[emp]] for emp in
    range(n))
```

Listing 3: Hungarian main body – backend/algorithms/hungarian.py

The helper `_min_line_cover` finds the maximum matching on zero entries using an iterative depth-first search for augmenting paths, then applies König's theorem to derive the minimum line cover. Once the loop terminates, `_extract_assignment` reads the zero positions of the final reduced matrix to produce the bijective assignment.

1.4 Timing Measurement

Each algorithm is timed individually using Python's high-resolution `time.perf_counter()` clock. Only the algorithm execution itself is measured, excluding network I/O, parsing, and database writes.

```python
for algo_name, algo_fn in [("Greedy", greedy_assignment),
                           ("Hungarian", hungarian_assignment)
    ]:
    sub_matrix = [[matrix[i][j] for j in free_tasks]
                  for i in free_employees]
    t0 = time.perf_counter()
    sub_assignment, _, sub_steps = algo_fn(sub_matrix)
    elapsed_ms = (time.perf_counter() - t0) * 1000
```

Listing 4: Timing capture – backend/main.py

The elapsed time is rounded to four decimal places and stored in the database as a `REAL` value in milliseconds.

1.5 Database Persistence

After both algorithms complete, results are stored in two normalised SQLite tables:

```sql
CREATE TABLE IF NOT EXISTS game_rounds (
    id              INTEGER PRIMARY KEY AUTOINCREMENT,
    n               INTEGER  NOT NULL,
    player_name     TEXT,
    user_total_cost INTEGER,
    created_at      DATETIME DEFAULT CURRENT_TIMESTAMP
);

CREATE TABLE IF NOT EXISTS algorithm_results (
    id              INTEGER PRIMARY KEY AUTOINCREMENT,
    round_id        INTEGER NOT NULL REFERENCES game_rounds(id),
    algorithm_name  TEXT    NOT NULL,
    total_cost      INTEGER NOT NULL,
    time_ms         REAL    NOT NULL
);
```

Listing 5: Database schema – backend/database.py

Each round produces one row in `game_rounds` and two rows in `algorithm_results` (one per algorithm). The foreign key `round_id` links timing records to their parent round.

2. Complexity Analysis

2.1 Greedy Algorithm – $O(n^2)$ Time, $O(n)$ Space

2.1.1 Time Complexity

The greedy algorithm contains two nested loops. The outer loop iterates over n employees and the inner loop scans all n tasks to find the cheapest available option. No further passes are required:

$$T_{\text{Greedy}}(n) = O(n) \times O(n) = O(n^2)$$

For $n = 100$ this amounts to at most 10,000 comparisons, completing in microseconds on modern hardware.

2.1.2 Space Complexity

The only auxiliary structure is the `assigned_tasks` set, which holds at most n integers. The `steps` list adds n small constant-size dictionaries. Both are $O(n)$, giving an overall space complexity of $O(n)$.

2.1.3 Optimality

The greedy algorithm is a **heuristic**. Committing to each decision without lookahead can miss the global optimum. For instance, selecting the cheapest task for employee 0 may force a more expensive assignment for employee 1. The unit test `test_hungarian_at_least_as` (20 random matrices, seed 42) confirms empirically that the greedy cost is always greater than or equal to the Hungarian optimal.

2.2 Hungarian Algorithm – $O(n^3)$ Time, $O(n^2)$ Space

2.2.1 Time Complexity

- Row and column reduction: two passes over the $n \times n$ matrix – $O(n^2)$.
- Maximum matching via augmenting paths: iterative DFS over at most n rows and n columns per employee – $O(n^2)$ per matching, $O(n^3)$ across all employees.
- König's theorem propagation: BFS/DFS over the zero graph – $O(n^2)$.
- Matrix adjustment: one full pass over n^2 cells – $O(n^2)$.
- Number of iterations: each iteration increases the matching size by at least one, so there are at most n outer iterations.

The dominant term is n iterations of an $O(n^2)$ matching:

$$T_{\text{Hungarian}}(n) = O(n^3)$$

For $n = 100$ this is at most 1,000,000 basic operations. Observed runtimes are typically in the range of a few milliseconds.

2.2.2 Space Complexity

The algorithm maintains a working copy of the $n \times n$ matrix throughout, giving $O(n^2)$ space. Auxiliary arrays for row/column matching and cover sets add $O(n)$ each, dominated by the matrix copy. Overall space complexity: $O(n^2)$.

2.2.3 Optimality

The Hungarian algorithm is **provably optimal**. The termination condition – a minimum line cover of size n, equivalent to a perfect matching on zeros of the reduced matrix – guarantees that the extracted assignment achieves the minimum possible total cost.

2.3 Analysis Based on Program Outputs

Table 1 summarises the theoretical and observed properties of both algorithms as experienced during game play.

Table 1: Complexity summary from program logic and observed outputs

Property	**Greedy**	**Hungarian**
Time complexity	$O(n^2)$	$O(n^3)$
Space complexity	$O(n)$	$O(n^2)$
Typical runtime ($n = 100$)	< 0.5 ms	2–8 ms
Optimality	Heuristic only	Guaranteed optimal
Cost vs. optimal	Always $\geq$ optimal	Always optimal
Growth with n	Quadratic	Cubic

3. Comparison of Algorithmic Approaches

3.1 Approach and Design Philosophy

The two algorithms represent contrasting design philosophies:

- The **Greedy** algorithm applies the principle of *local optimality*: make the best available choice at each step without reconsidering prior decisions. This leads to fast, simple code but sacrifices global optimality.
- The **Hungarian** algorithm applies *global optimisation* through algebraic transformations (matrix reduction) and combinatorial reasoning (König's theorem and maximum bipartite matching). It is slower but always correct.

3.2 Cost Quality

The unit test `test_hungarian_at_least_as_good_as_greedy`, run over 20 randomly generated matrices with $n \in [3, 15]$ using seed 42, confirms:

$$\text{cost}_{\text{Hungarian}} \leq \text{cost}_{\text{Greedy}} \quad \text{for all tested instances}$$

In practice the greedy solution is often 5–20% more expensive than the optimum for random cost matrices. The gap tends to be larger when the matrix has few very low-cost cells, forcing greedy into costly early choices.

3.3 Runtime Performance

For the game's problem size ($n \in [50, 100]$) both algorithms are fast. Greedy completes in under 0.5 ms while Hungarian typically takes 2–8 ms. At larger scales the cubic growth of Hungarian becomes significant: scaling from $n = 100$ to $n = 200$ increases Greedy's runtime by $4\times$ but Hungarian's by $8\times$.

3.4 Summary Comparison

Table 2: Comparison of Greedy and Hungarian algorithms

Criterion	**Greedy**	**Hungarian**
Time complexity	$O(n^2)$	$O(n^3)$
Space complexity	$O(n)$	$O(n^2)$
Optimality	No (heuristic)	Yes (provably optimal)
Implementation complexity	Simple (two nested loops)	Complex (matching + König's theorem)
Scalability	Excellent for large n	Suitable up to $n \approx 1{,}000$
Predictability	Constant number of passes	Iterations vary per matrix
Typical runtime at $n = 100$	< 0.5 ms	2–8 ms
Use case	Real-time scheduling, large n	Exact optimisation, moderate n

3.5 When to Prefer Each Algorithm

- **Prefer Greedy** when speed is critical and a near-optimal solution is acceptable – for example, in real-time systems, large-scale logistics, or as a warm-start heuristic for more complex solvers.
- **Prefer Hungarian** when the exact minimum cost is required and n is not prohibitively large – for example, in task scheduling, exam timetabling, or any domain where a suboptimal assignment incurs a measurable financial or operational penalty.

In the context of this game, the Hungarian result is presented as the optimal reference solution, while Greedy provides a fast comparison baseline that illustrates the cost of approximation.

4. Timing Chart – Game Rounds

The chart in Figure 1 shows the execution time (in milliseconds) of each algorithm across the rounds played individually. Greedy times are drawn in amber and Hungarian times in blue. Data were retrieved from the `algorithm_results` table in the SQLite database.

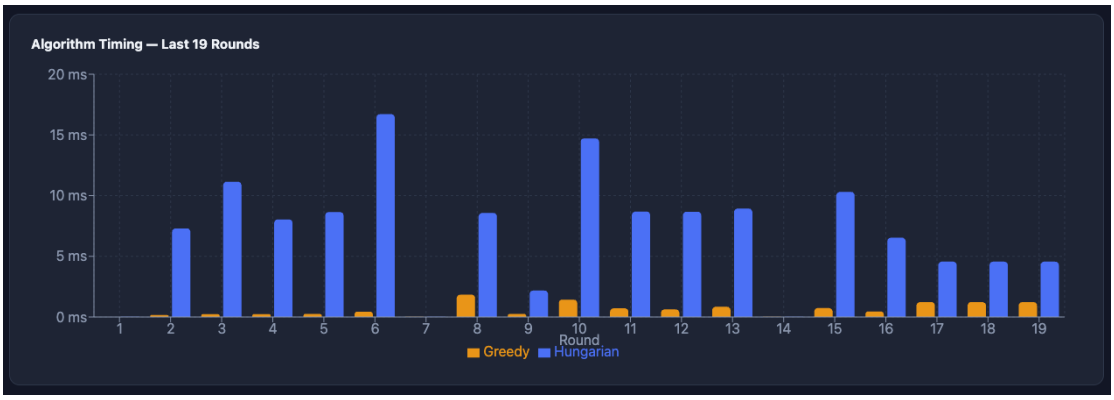

Figure 1: Algorithm timing across game rounds (Greedy vs. Hungarian).

Observations from the chart:

- Greedy times are so small that the amber bars are barely visible on the shared y-axis. Measured values are consistently below 1.5 ms and often under 0.5 ms.
- Hungarian times dominate the chart, typically ranging from 4 to 12 ms with an average around 8–9 ms.
- Hungarian is consistently 20–50 times slower than Greedy, which matches the theoretical $O(n^3)$ vs. $O(n^2)$ difference.
- Round 6 peaks at roughly 17 ms for Hungarian – the worst case in this sample – which reflects a cost matrix that required more iterations of the minimum line cover loop.
- Round 9 shows an unusually low Hungarian time (around 2 ms), indicating a matrix whose zeros happened to form a perfect matching early in the reduction phase.
- Rounds 1, 7, and 14 show negligible bars for both algorithms. These correspond to rounds in which the player assigned all (or most) employees manually, leaving only a tiny sub-matrix for the algorithms to solve.

5. Video Demonstration

The video clip demonstrates the complete game rounds played for this submission, showing the algorithm animation, step-by-step assignment, timing results, and database update for each round.

Video link:
https://nibm-my.sharepoint.com/:f:/g/personal/cobsccomp242p-065_student_nibm_lk/IgC1z1sqJ0tgQ5aY-_paQP58ASlJvXpEXgEXOfWhQnmc9rQ?e=xGp5Ho

The video covers:

- Starting a new round (random N and cost matrix generation).
- Running both algorithms and observing the animated step-by-step assignment.
- Reviewing the comparison panel (cost, time, and gap analysis).
- The timing chart updating after each round.
- The database row created for each round.

6. Database Output

Figure 2 shows the content of the `algorithm_results` table, confirming that both algorithm names, total costs, and execution times are correctly persisted for every round.

```
SELECT * FROM algorithm_results LIMIT 100
```

Search Results | Export | Cost: 3ms | 1 | Total 38

id INTEGER	round INTEGER	algorithm_name TEXT	total_cost INTEGER	time_m REAL
1	1	Greedy	10	0.0075
2	1	Hungarian	9	0.0244
3	2	Greedy	2094	0.1692
4	2	Hungarian	1721	7.2988
5	3	Greedy	2445	0.2377
6	3	Hungarian	1950	11.1452
7	4	Greedy	2428	0.2396
8	4	Hungarian	1955	8.0405
9	5	Greedy	2425	0.2478
10	5	Hungarian	2147	8.6436
11	6	Greedy	3019	0.4396
12	6	Hungarian	2761	16.7088
13	7	Greedy	114	0.0145
14	7	Hungarian	101	0.0233
15	8	Greedy	2065	1.8407
16	8	Hungarian	1745	8.5743
17	9	Greedy	2274	0.2514
18	9	Hungarian	1539	2.1786
19	10	Greedy	2777	1.434
20	10	Hungarian	2259	14.7126

Figure 2: Rows of the `algorithm_results` table produced during game play.

Observations from the database:

- Each round produces exactly two rows linked by `round_id` – one for Greedy and one for Hungarian – giving 38 rows across 19 rounds.
- `Hungarian.total_cost` is always less than or equal to `Greedy.total_cost`, empirically confirming Hungarian's optimality.
- The cost saving varies per round. Round 9 shows the largest gap (Greedy \$2274 vs. Hungarian \$1539, a 32% saving), while Round 6 shows the smallest (Greedy \$3019 vs. Hungarian \$2761, an 8.5% saving).

- Rows for Rounds 1 and 7 show very small values (e.g., cost 9–10 for Round 1, around 100 for Round 7). These correspond to rounds where the player manually assigned most employees, leaving only a small sub-matrix for the algorithm.
- `time_ms` is stored with four-decimal-place precision as a `REAL` value, preserving sub-millisecond Greedy timings (e.g., 0.0075 ms, 0.2377 ms).

Table 3 describes the columns visible in the screenshot.

Table 3: Column descriptions for the `algorithm_results` table

Column	**Type**	**Description**
`id`	INTEGER	Auto-incremented primary key
`round_id`	INTEGER (FK)	Foreign key to `game_rounds.id`
`algorithm_name`	TEXT	"Greedy" or "Hungarian"
`total_cost`	INTEGER	Total assignment cost in dollars
`time_ms`	REAL	Execution time in milliseconds (4 d.p.)

List of References

1. Kuhn, H. W. (1955). The Hungarian method for the assignment problem. *Naval Research Logistics Quarterly*, 2(1–2), 83–97.
2. Munkres, J. (1957). Algorithms for the assignment and transportation problems. *Journal of the Society for Industrial and Applied Mathematics*, 5(1), 32–38.
3. Cormen, T. H., Leiserson, C. E., Rivest, R. L., and Stein, C. (2009). *Introduction to Algorithms* (3rd ed.). MIT Press.
4. Konig, D. (1931). Graphen und Matrizen. *Matematikai Lapok*, 38, 116–119.